3D-Printed Ducted-Fan Quadcopter

I designed the airframe in CAD, 3D-printed the main structure, assembled the physical prototype, and designed a custom STM32 flight-controller PCB for the same platform.
ALL FILES : GitHub Repository



WHAT THIS PROJECT SHOWS

- Mechanical CAD for a Modular ducted-fan quadcopter frame.
- 3D-printed parts assembled into a physical prototype.
- Custom KiCad flight-controller PCB from ground up.
- Shows a complete hardware development workflow from design to testing.

CURRENT STATUS

Mechanical prototype built. Custom PCB designed. Next step is board fabrication, bring-up, firmware, and controlled flight testing.

SECTIONS

CAD / Mechanical Design

Fusion 360 model, engineering drawing, and main geometry.

3D-Printed Assembly

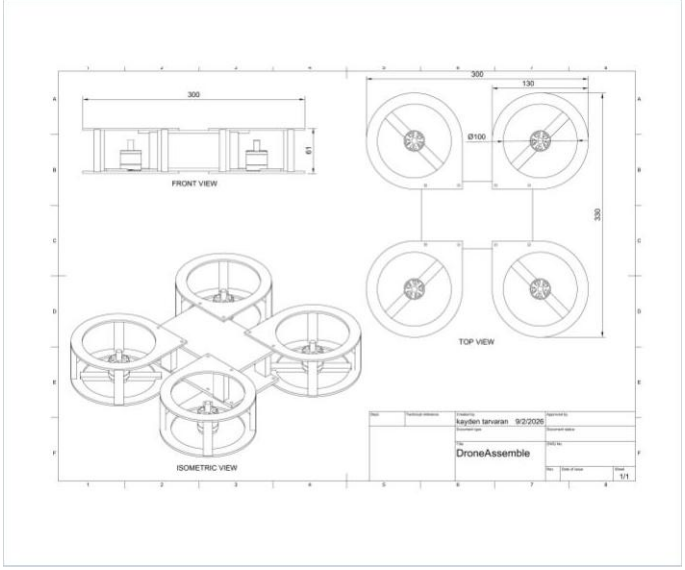
Printed and tested components, hardware fitment, and motor integration.

Custom PCB

STM32F4 controller board, sensors, power management, and I/O.

Airframe CAD and Assembly Drawing





MAIN DIMENSIONS

300 mm
OVERALL WIDTH

330 mm
OVERALL LENGTH

61 mm
OVERALL HEIGHT

Ø100 mm
DUCT OPENING

DESIGN NOTES

The frame uses four ducted propeller modules connected to a central body.

The frame was designed to support stability and navigation testing by using four 2300KV brushless motors connected to 10cm dual blade propellors. A strong focus was placed on modular design to make the system easy to manufacture, modify, and upgrade.

TOOLS USED

Fusion 360 for 3D modeling and technical drawings, Ulti maker Cura for slicing and G-code generation, and an Ender 3 for 3D printing.

From Printed Parts to Prototype





LEFT: PART LAYOUT

The printed parts were laid out before assembly to check that the ducts, plates, supports, and motor modules matched the CAD plan.

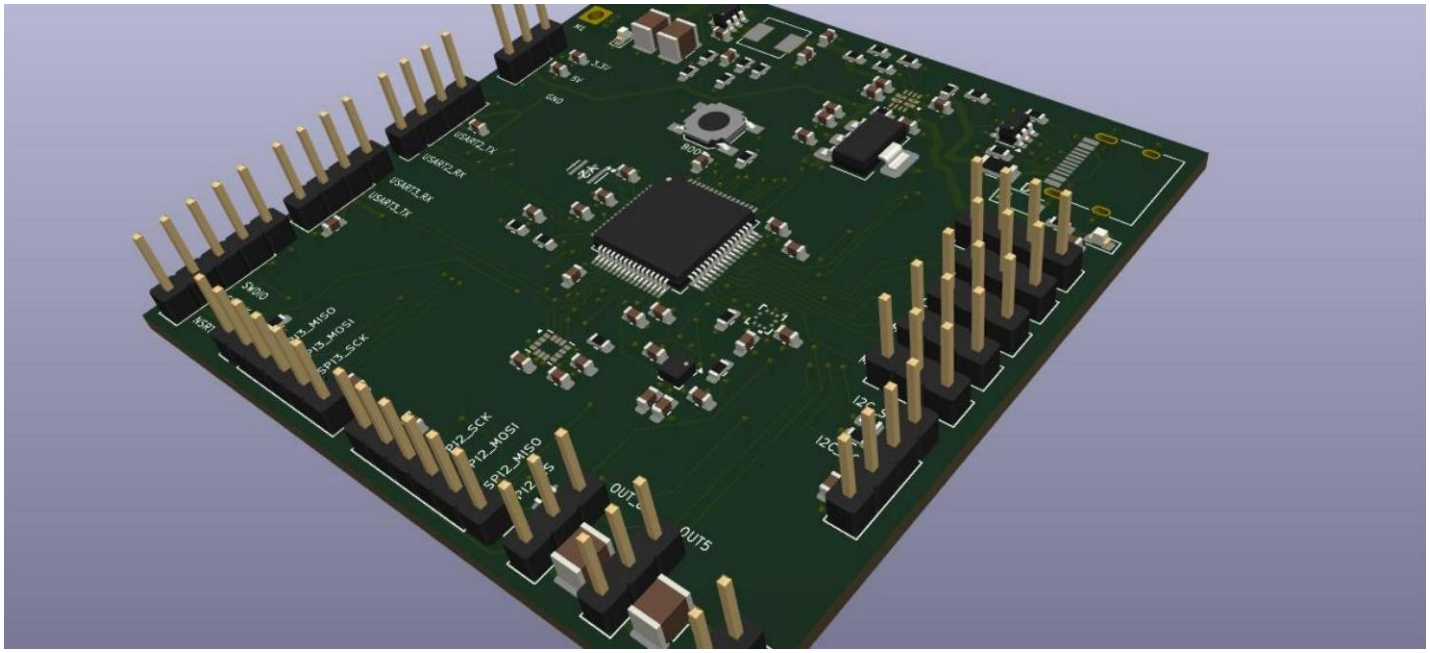
RIGHT: ASSEMBLED FRAME

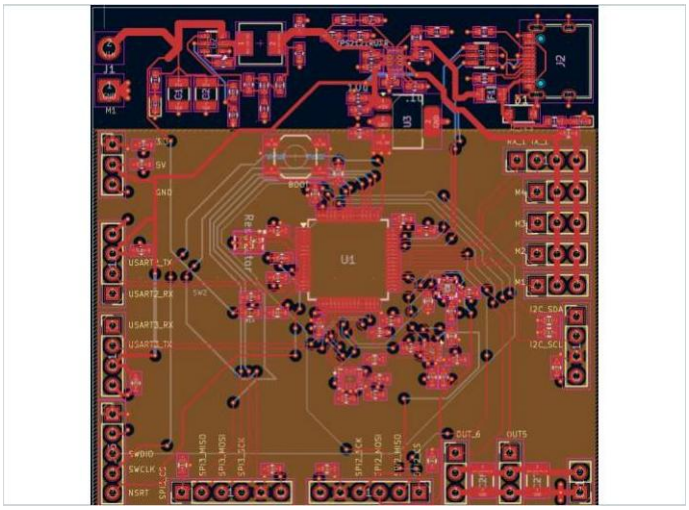
The duct assembly was screwed to the main body, while the pillars were glued in place to secure the structure.

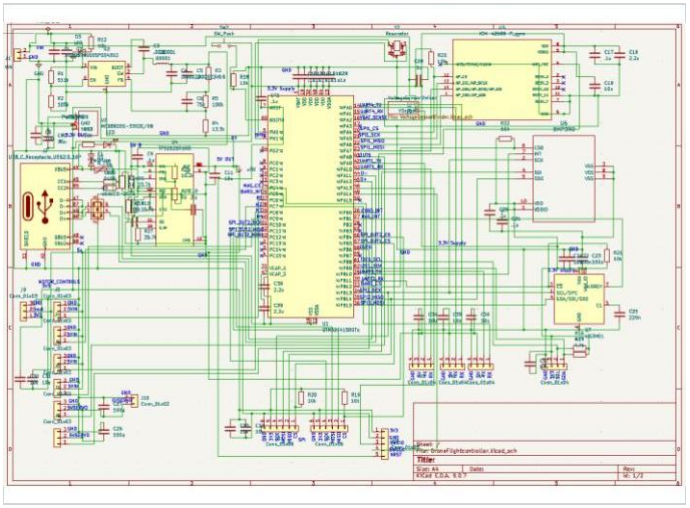
WHAT I LEARNED FROM THIS BUILD

Design choices were driven by the manufacturing process. Using a basic 3D printer required the frame to be split into smaller, more printable parts to improve reliability and simplify reprints.

Custom STM32 Flight-Controller PCB







BOARD CHOICES

- STM32F415RGT6 MCU as the main controller.
- ICM-42688-P gyro/IMU, BMP390 barometer, and LIS2MDL magnetometer.
- 5 V buck regulator, 3.3 V LDO rail, and USB-C / battery power path.
- SPI, I²C, UART, PWM motor outputs, SWD programming, and external I/O headers.

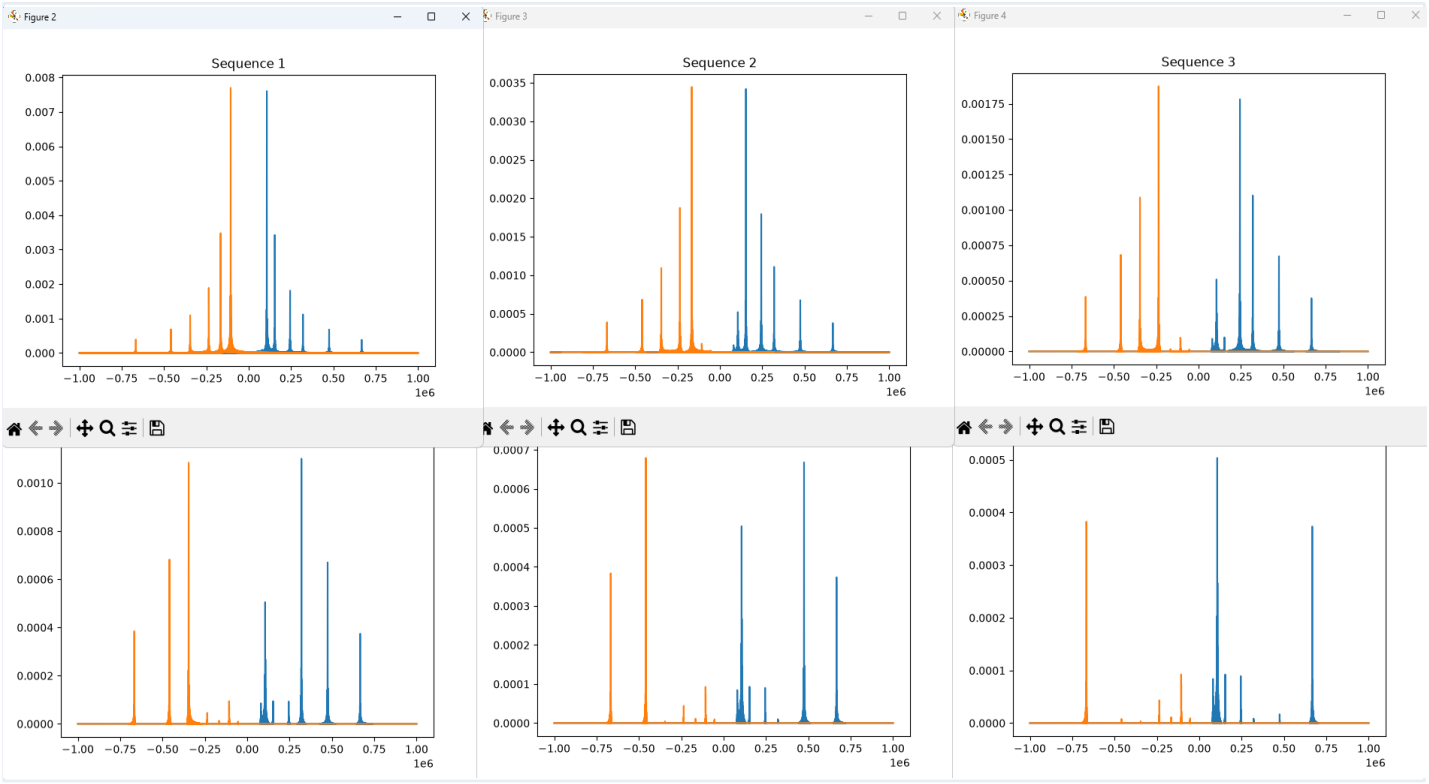
NEXT REVISION / FUTURE WORK

Order and assemble the PCB, Configure onboard sensor telemetry and functionality, develope firmware, incorporate external sensors for autonomous / Intelligent navigation.

Python FMCW Radar Simulator

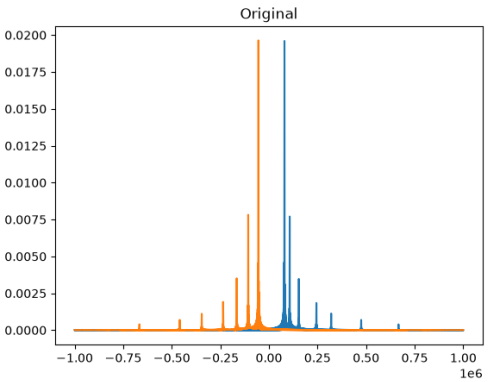
Developed a Python FMCW dual chirp radar simulation and signal-processing pipeline to estimate target range, radial velocity, and RCS from simulated beat spectra.
All Files: Git Repository

SUCCESSIVE INTERFERENCE CANCELLATION RUN



SIC sequence plots show the strongest target waveform and its associated spectral leakage being modeled and removed, exposing weaker target returns in the residual spectrum.

REFERENCE SPECTRA



RESULT

6 of 7 simulated targets were accurately recovered. The final SIC iteration locked onto residual spectral energy, which motivates CFAR or an edited SIC algorithm.

RECOVERED TARGET ESTIMATES

```
Calculated Target 1 : 5.00 25.00 1.00 Real Target: (5.0, 25, 1.0) Error: 0.00 0.02 0.31
Calculated Target 2 : 8.00 0.05 0.96 Real Target: (8.0, 0, 1.0) Error: 0.02 0.05 3.89
Calculated Target 3 : 12.00 -11.98 0.98 Real Target: (12.0, -12, 1.0) Error: 0.01 0.19 1.85
Calculated Target 4 : 18.00 7.96 1.41 Real Target: (18.0, 8, 1.5) Error: 0.00 0.55 6.04
Calculated Target 5 : 25.00 -24.99 1.91 Real Target: (25.0, -25, 2.0) Error: 0.00 0.03 4.64
Calculated Target 6 : 35.00 14.99 2.75 Real Target: (35.0, 15, 3.0) Error: 0.00 0.07 8.27
Calculated Target 7 : 29.52 -546.26 5.57 Real Target: (50.0, 0, 4.0) Error: 40.97 546.26 39.17
```

PIPELINE: Target scene → FMCW chirps → dechirp → frequency analysis → peak detection → SIC residual → recovered targets

TOOLS USED: Python, NumPy, SciPy, Matplotlib

UART Transmitter/Receiver

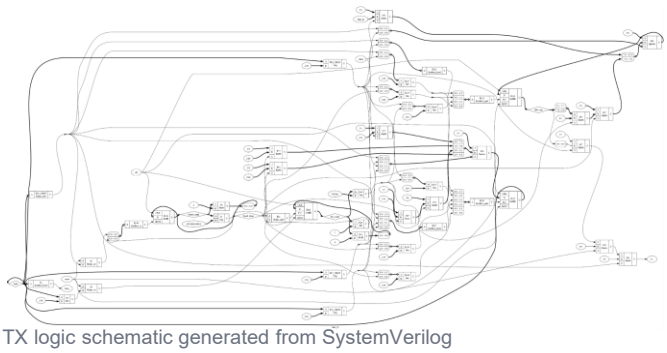
Built a configurable 8N1 UART core in SystemVerilog with FSM-based TX/RX logic, baud timing, and loopback verification.

All Files: GitHub Repository

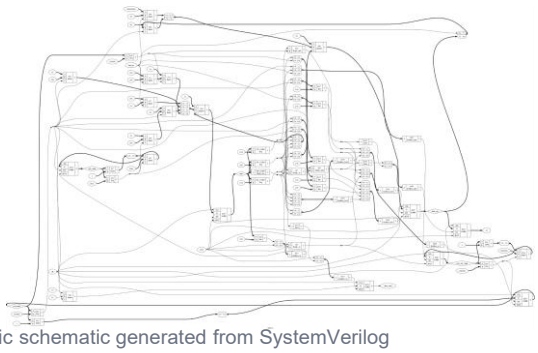
UART DATA PATH

TX DATA
8-bit parallel
—
TX FSM
start / data / stop
—
SERIAL LINE
tx
—
RX FSM
16× sample
—
RX DATA
8-bit parallel

SYNTHESIS / RTL SCHEMATIC VIEWS



TX logic schematic generated from SystemVerilog



RX logic schematic generated from SystemVerilog

CORE DESIGN

- 8N1 UART frame: start, 8 data bits, stop.
- Configurable clock frequency and baud rate.
- Separate TX and RX state machines.

VERIFICATION

- TX output wired directly into RX input.
- Loopback test sends A5, 00, FF, 55, 48.
- Receive data checked when rx_valid asserts.

TOOLS USED

- SystemVerilog
- Icarus Verilog testbench
- Yosys RTL schematic generation

```
PS C:\WORK\SystemVerilog\UART> vvp sim
PASS: sent=a5 received=a5
PASS: sent=00 received=00
PASS: sent=ff received=ff
PASS: sent=55 received=55
PASS: sent=48 received=48
uart_tb.sv:78: $finish called at 429485000 (1ps)
```

PROJECT STATUS

Simulation-level UART transmit/receive behavior completed. Next step: FPGA hardware loopback and waveform capture.